

Лекция 04. Поиск пути в графе.

Задача о кратчайшем пути

Задача о кратчайшем пути — задача поиска самого короткого пути(цепи) между двумя точками (вершинами) на графе, в которой минимизируется сумма весов рёбер, составляющих путь.

Постановки задачи

- Задача о кратчайшем пути в заданный пункт назначения
- Задача о кратчайшем пути между заданной парой вершин
- Задача о кратчайшем пути между всеми парами вершин

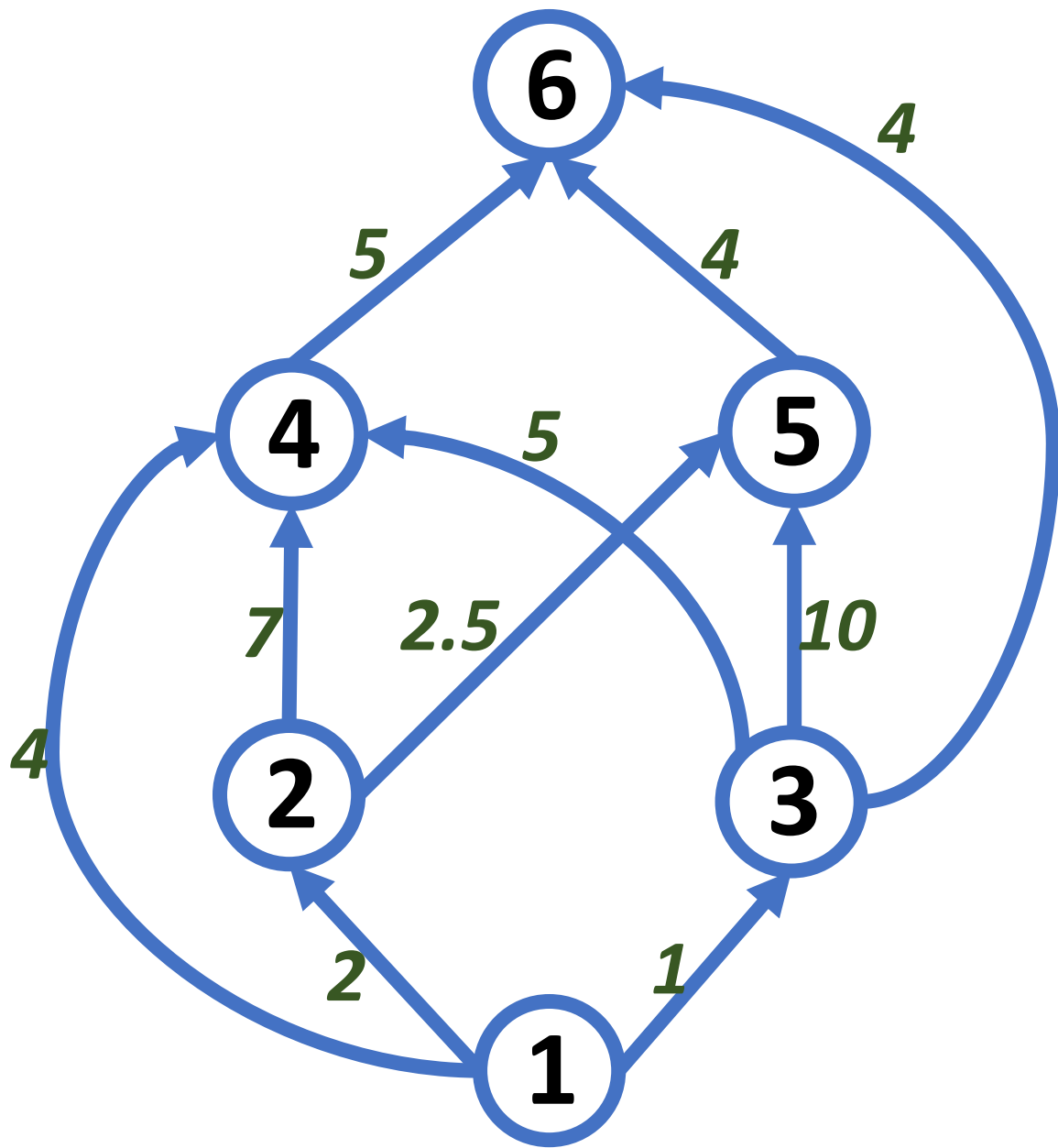
Алгоритм Дейкстры

- Находит кратчайший путь от одной из вершин графа до всех остальных. Алгоритм работает только для графов без ребер отрицательного веса.

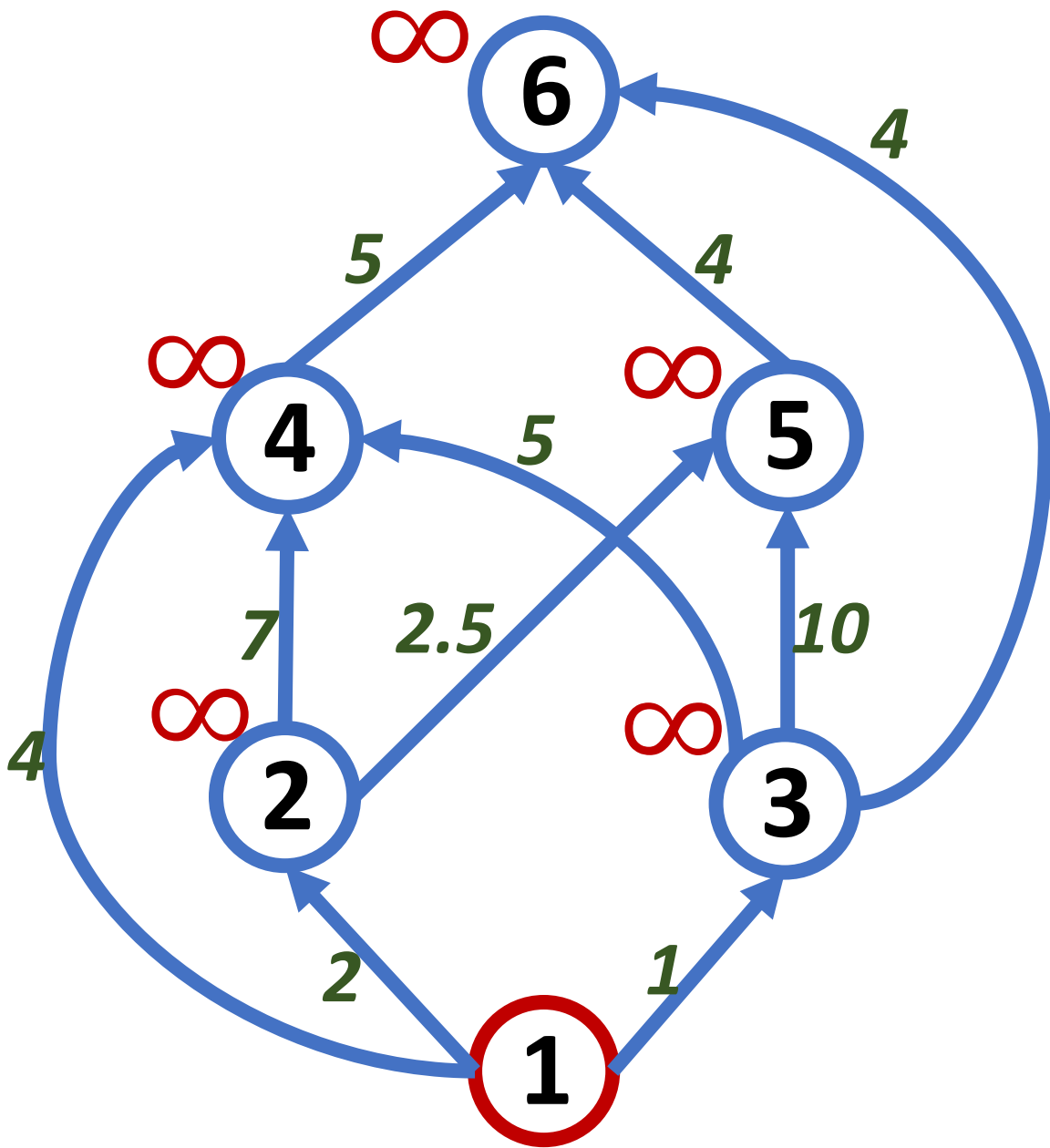
Алгоритм Дейкстры

- Шаг 1. Всем вершинам, за исключением первой, присваивается *вес* равный бесконечности, а первой вершине – 0.
- Шаг 2. Все вершины не выделены.
- Шаг 3. Первая *вершина* объявляется текущей.
- Шаг 4. *Вес* всех невыделенных вершин пересчитывается по формуле: *вес* невыделенной вершины есть минимальное число из старого *веса* данной вершины, суммы *веса* текущей вершины и *веса ребра*, соединяющего текущую вершину с невыделенной.
- Шаг 5. Среди невыделенных вершин ищется *вершина* с минимальным *весом*. Если таковая не найдена, то есть *вес* всех вершин равен бесконечности, то *маршрут* не существует. Следовательно, *выход*. Иначе, текущей становится найденная *вершина*. Она же выделяется.
- Шаг 6. Если текущей вершиной оказывается конечная, то *путь* найден, и его *вес* есть *вес* конечной вершины.
- Шаг 7. Переход на шаг 4.

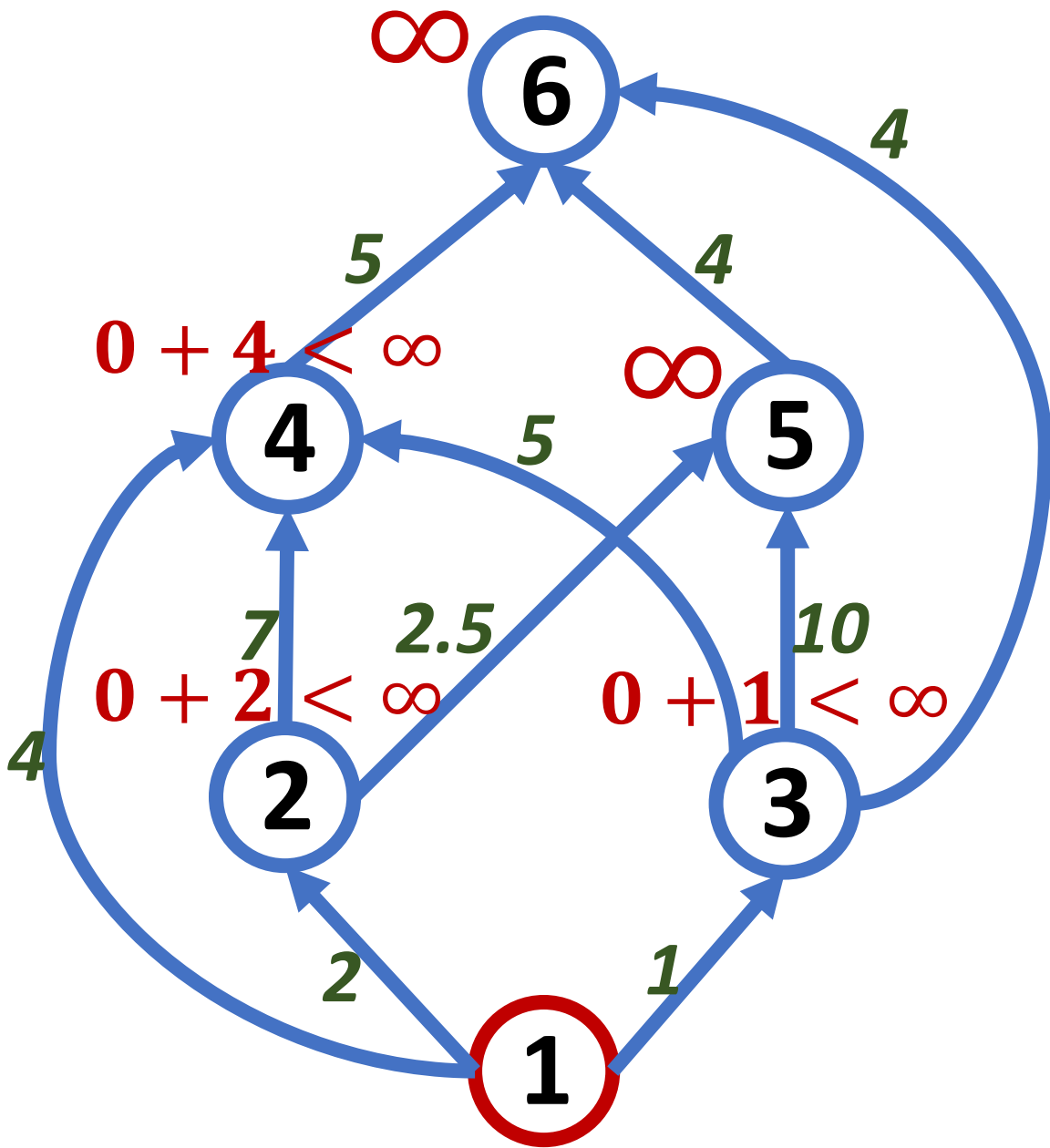
Найти кратчайший путь из вершины 1 в вершину 6



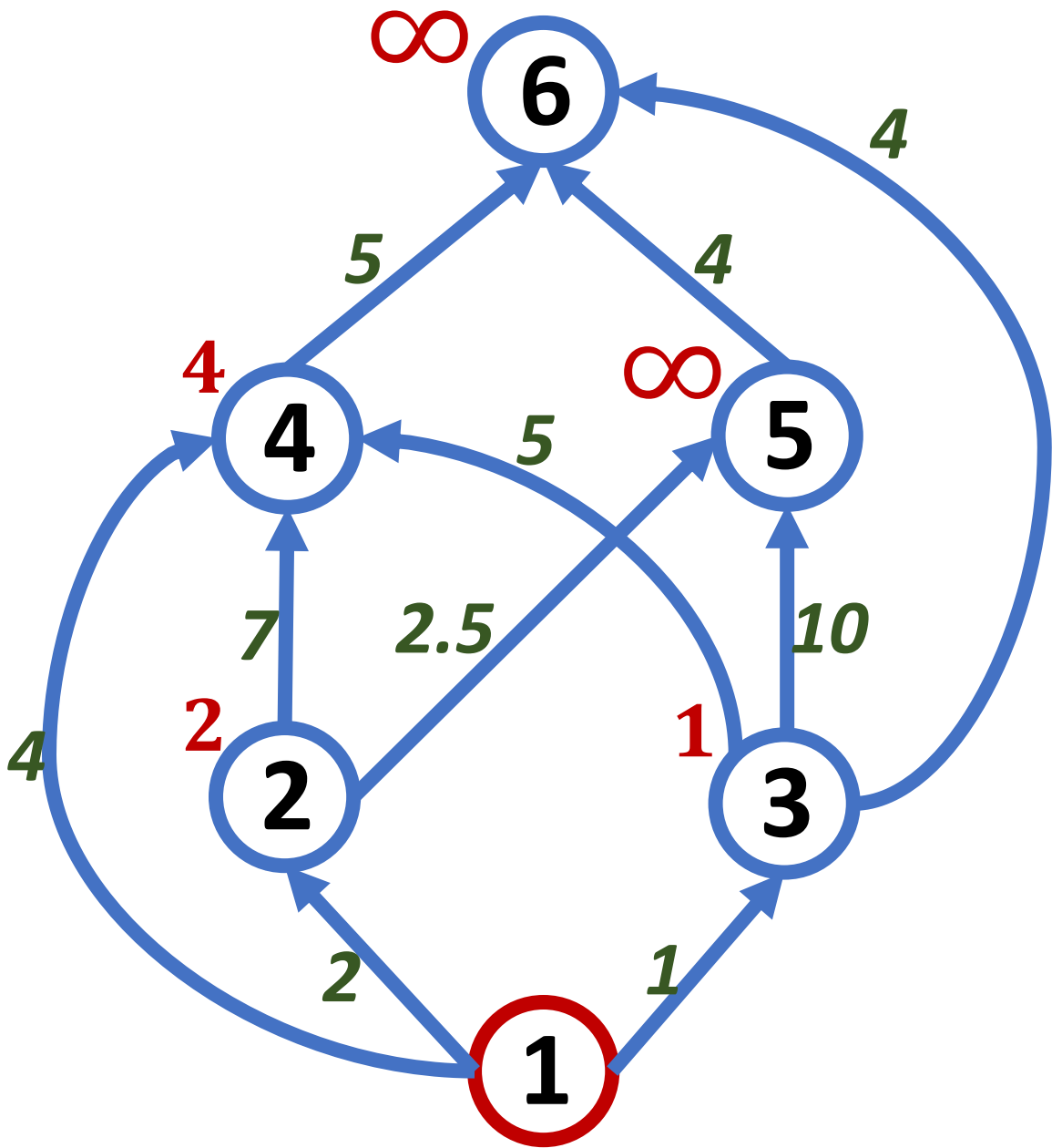
1	2	3	4	5	6
0	0	0	0	0	0



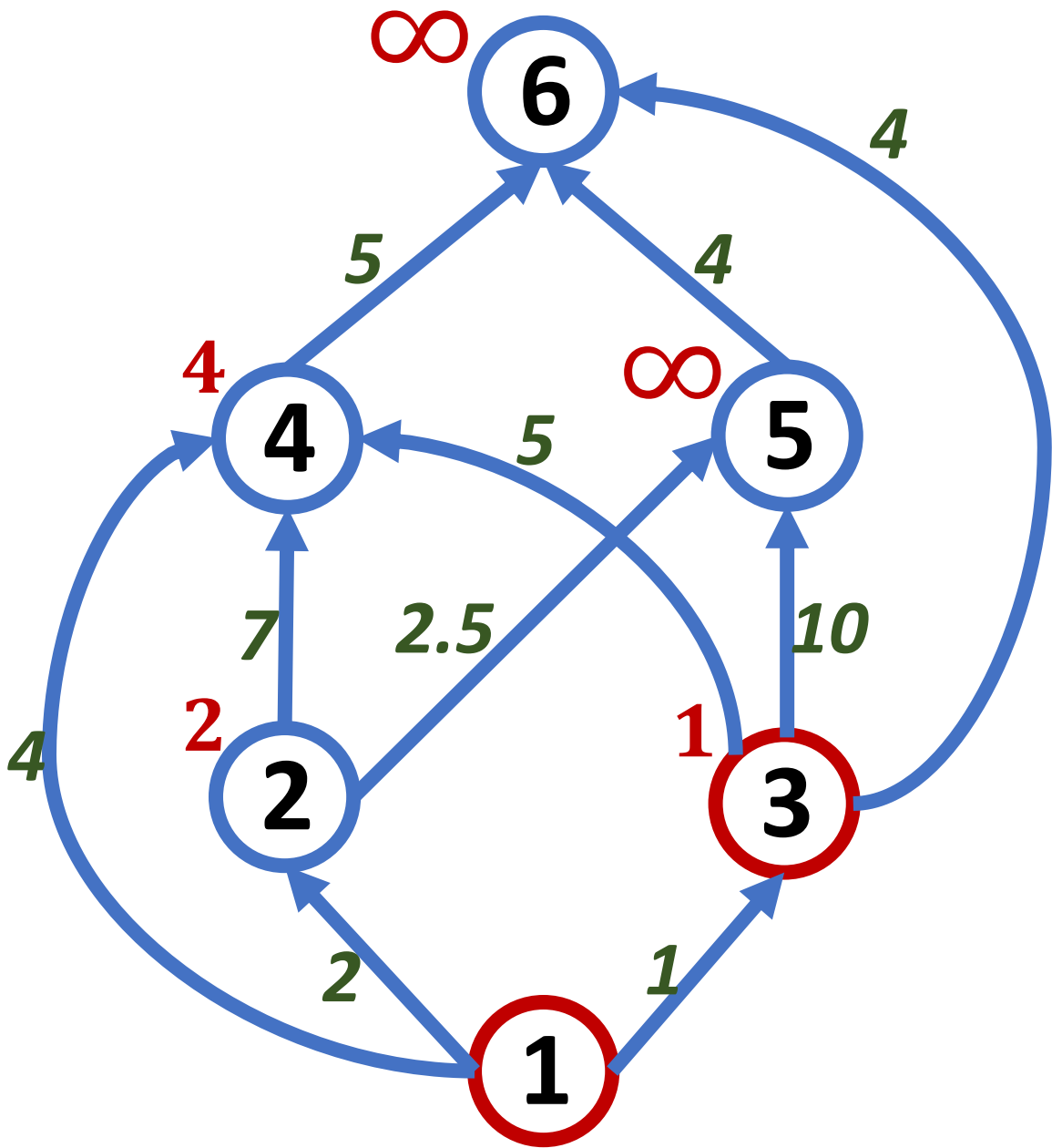
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
0	0	0	0	0	0



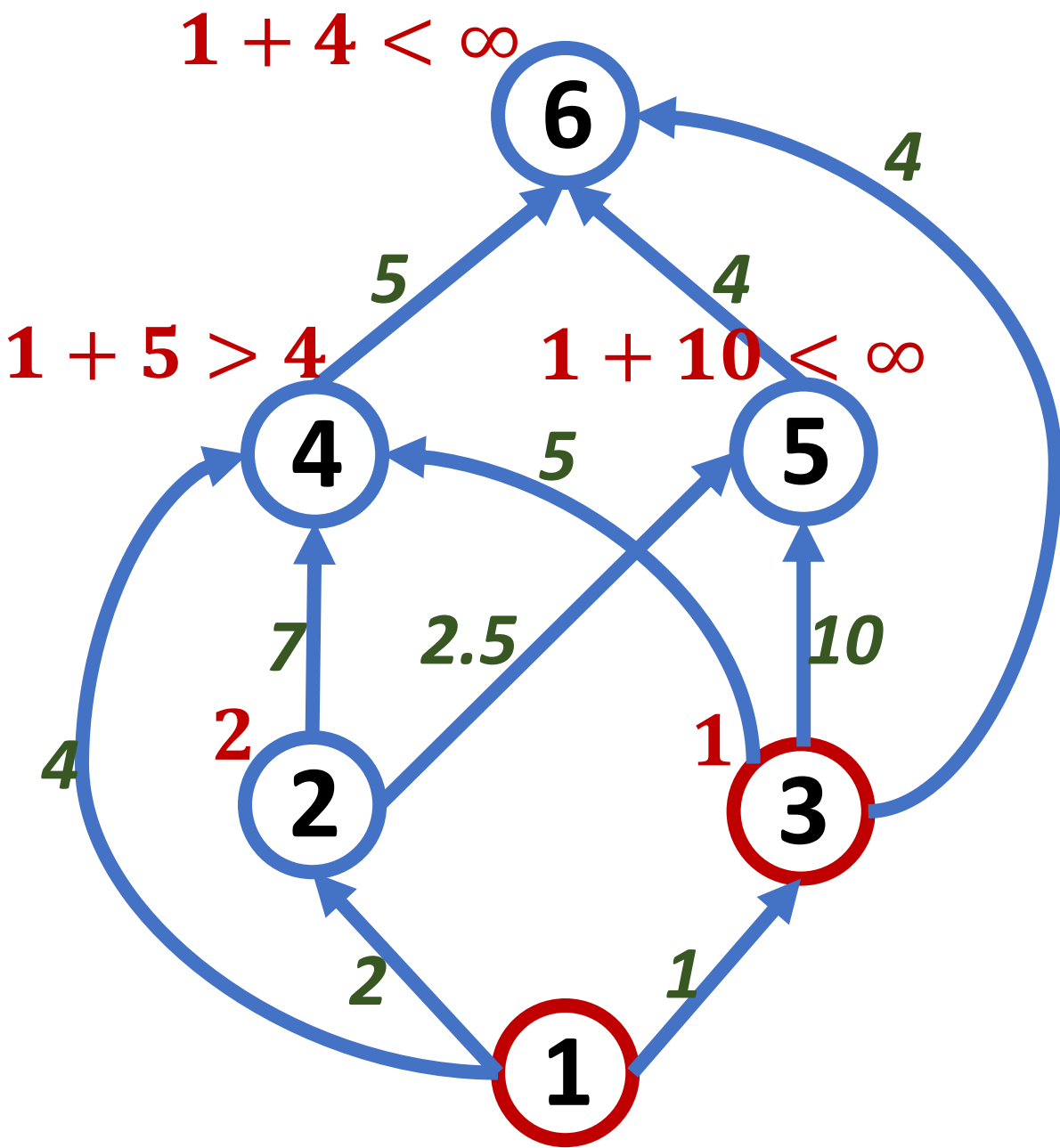
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	$0+2 < \infty$	$0+1 < \infty$	$0+4 < \infty$		
0	0	0	0	0	0



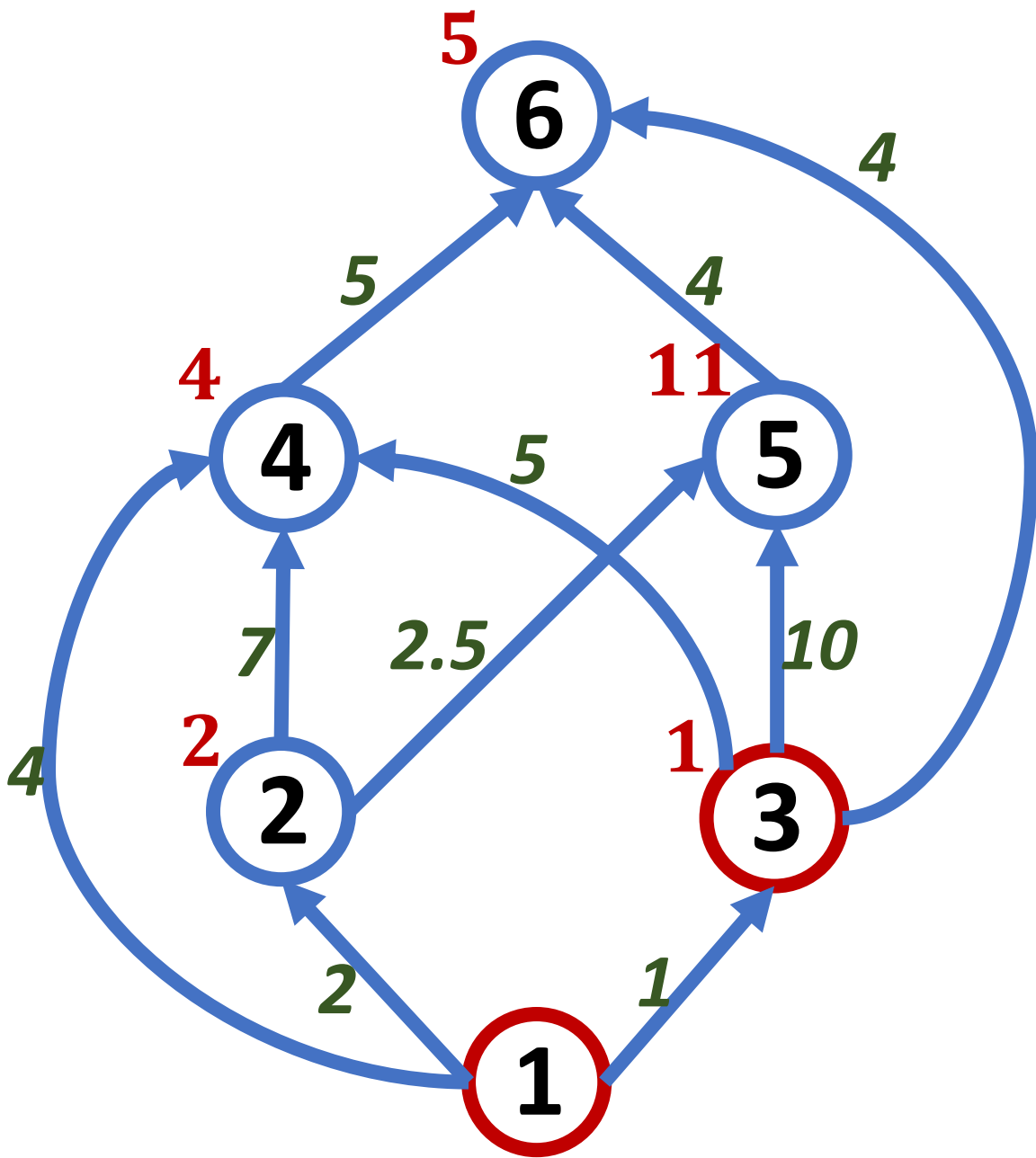
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
0	1	1	1	0	0



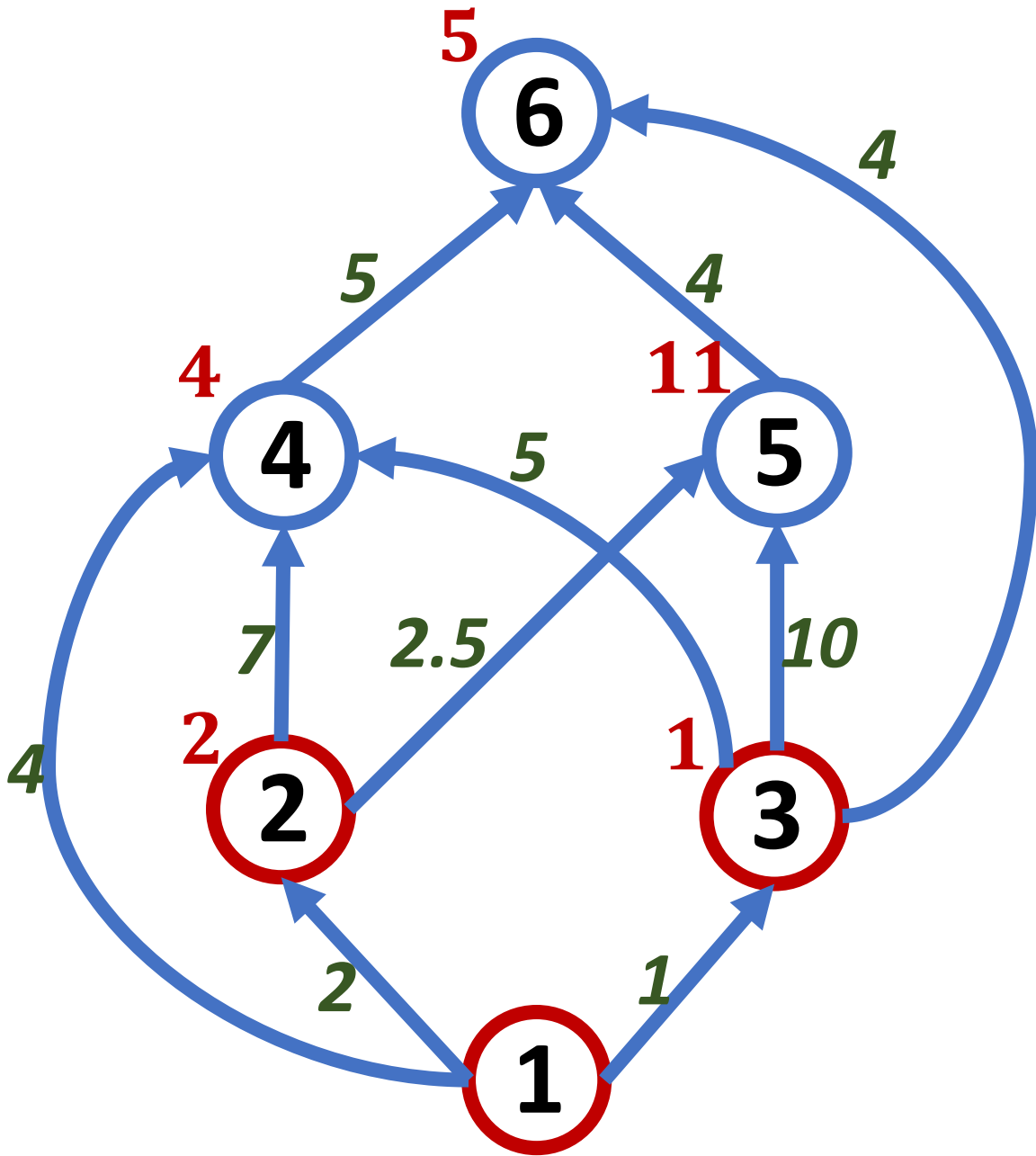
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
0	1	1	1	0	0



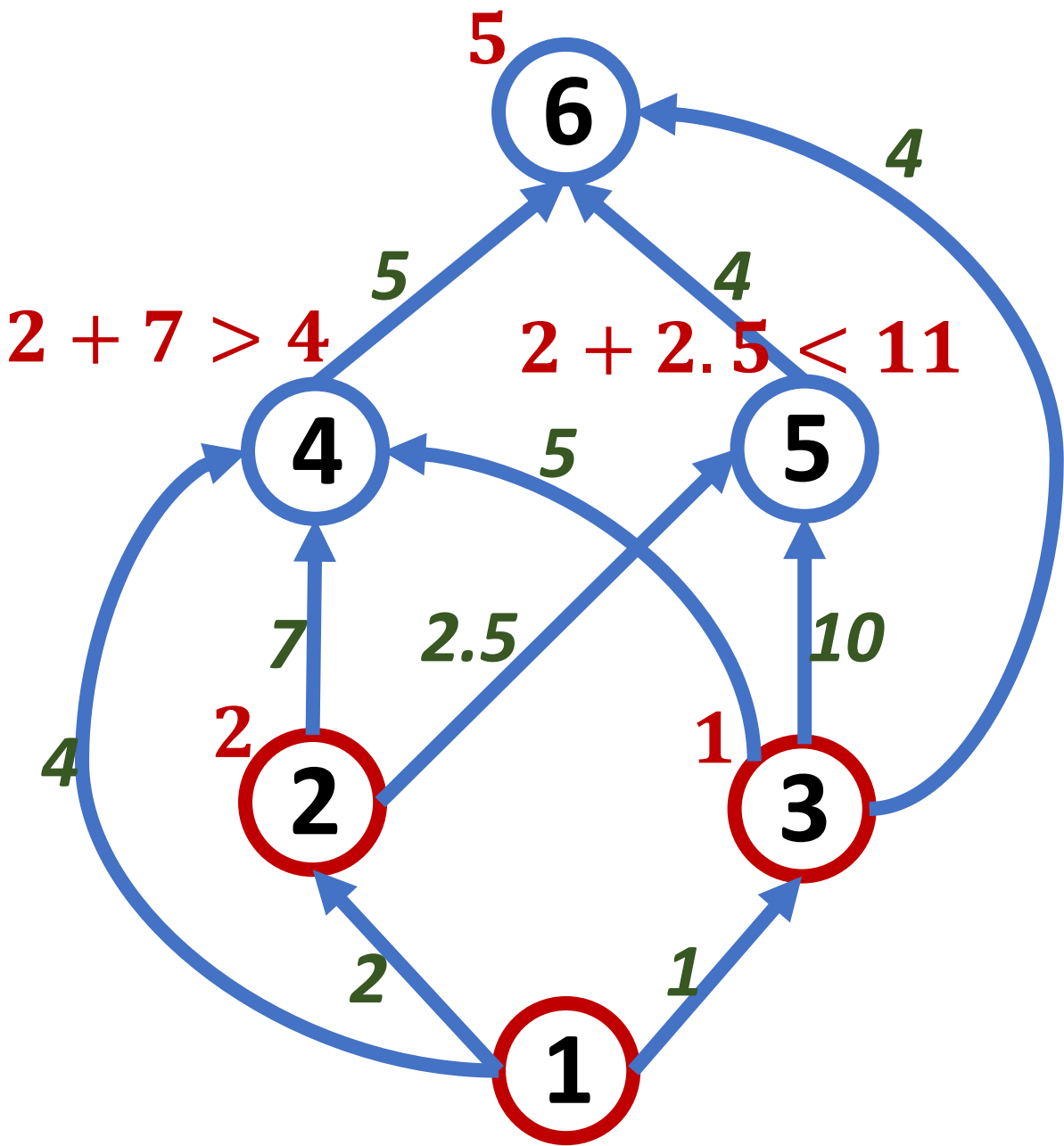
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		$1+5>4$	$1+10<\infty$	$1+4<\infty$
0	1	1	1	0	0



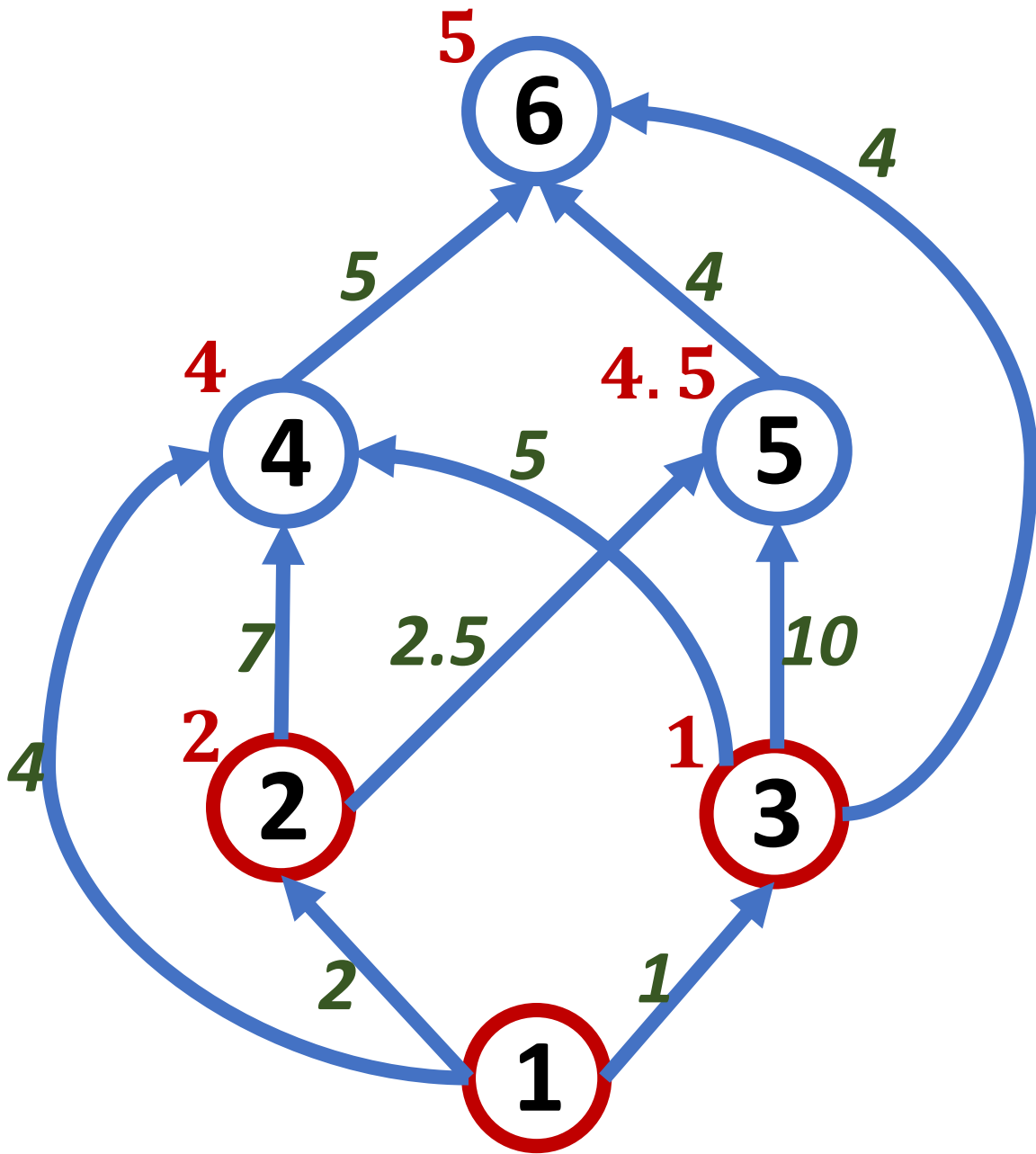
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
0	1	1	1	3	3



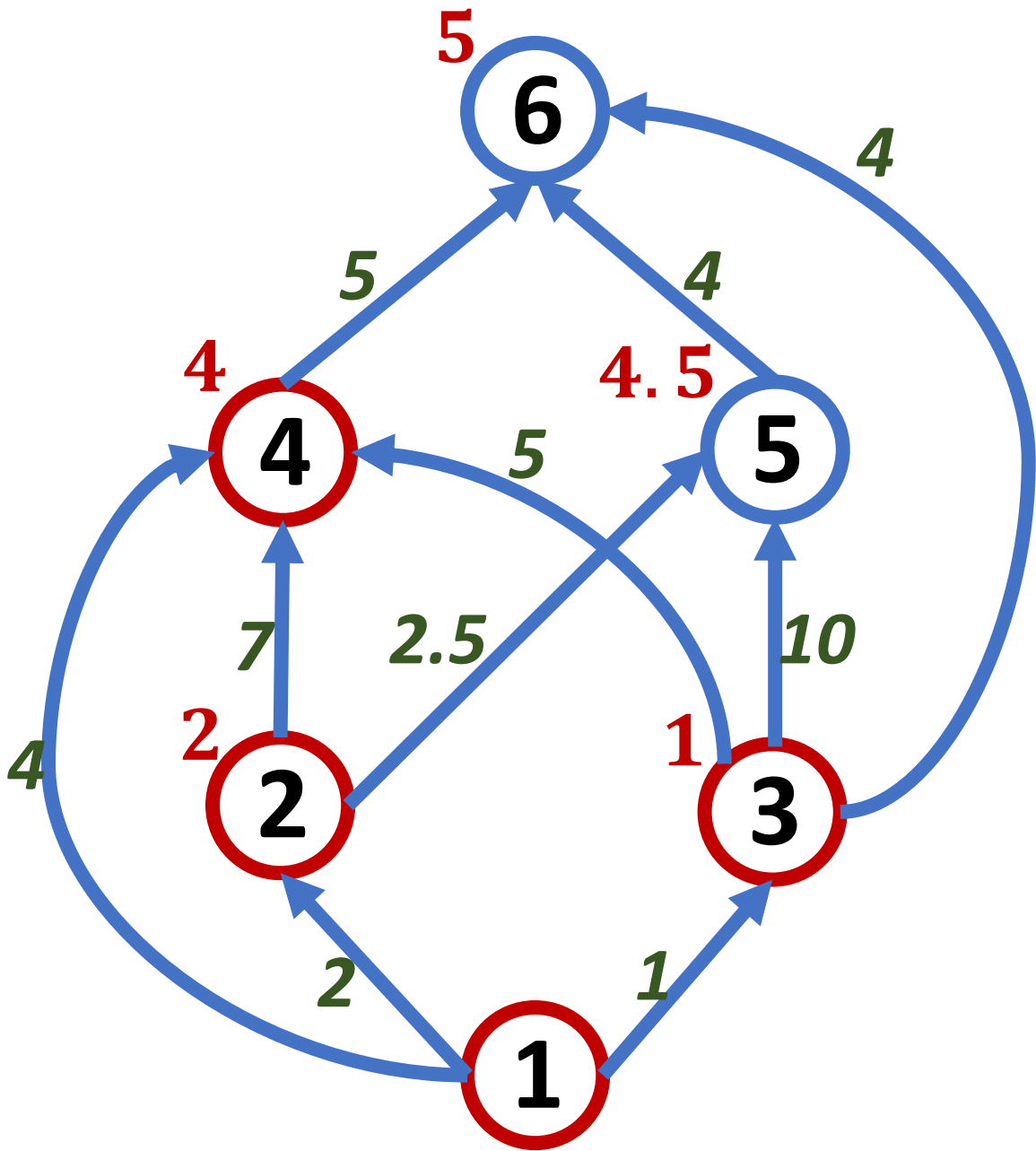
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
0	1	1	1	3	3



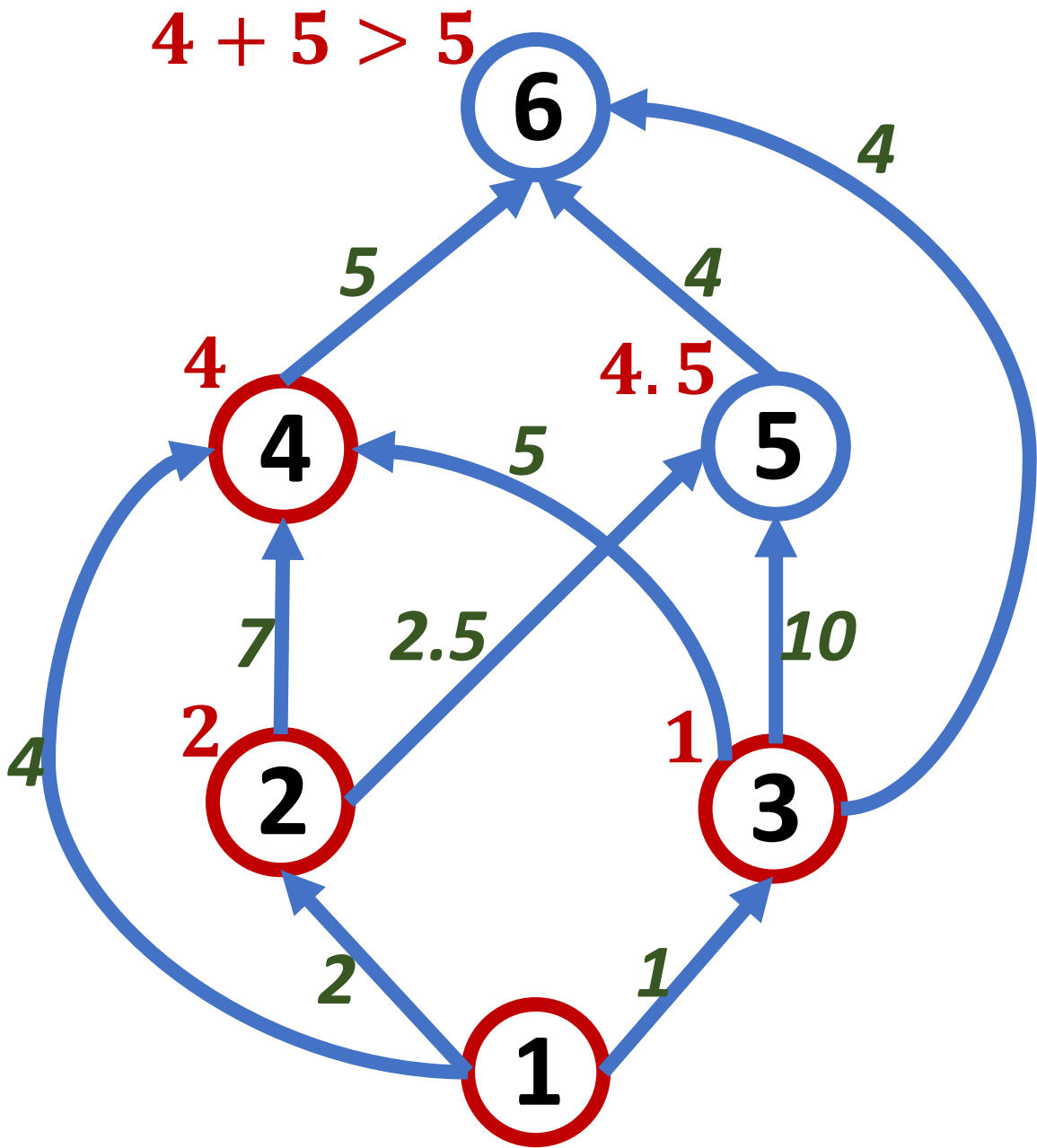
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			$2+7>4$	$2+2.5<11$	5
0	1	1	1	3	3



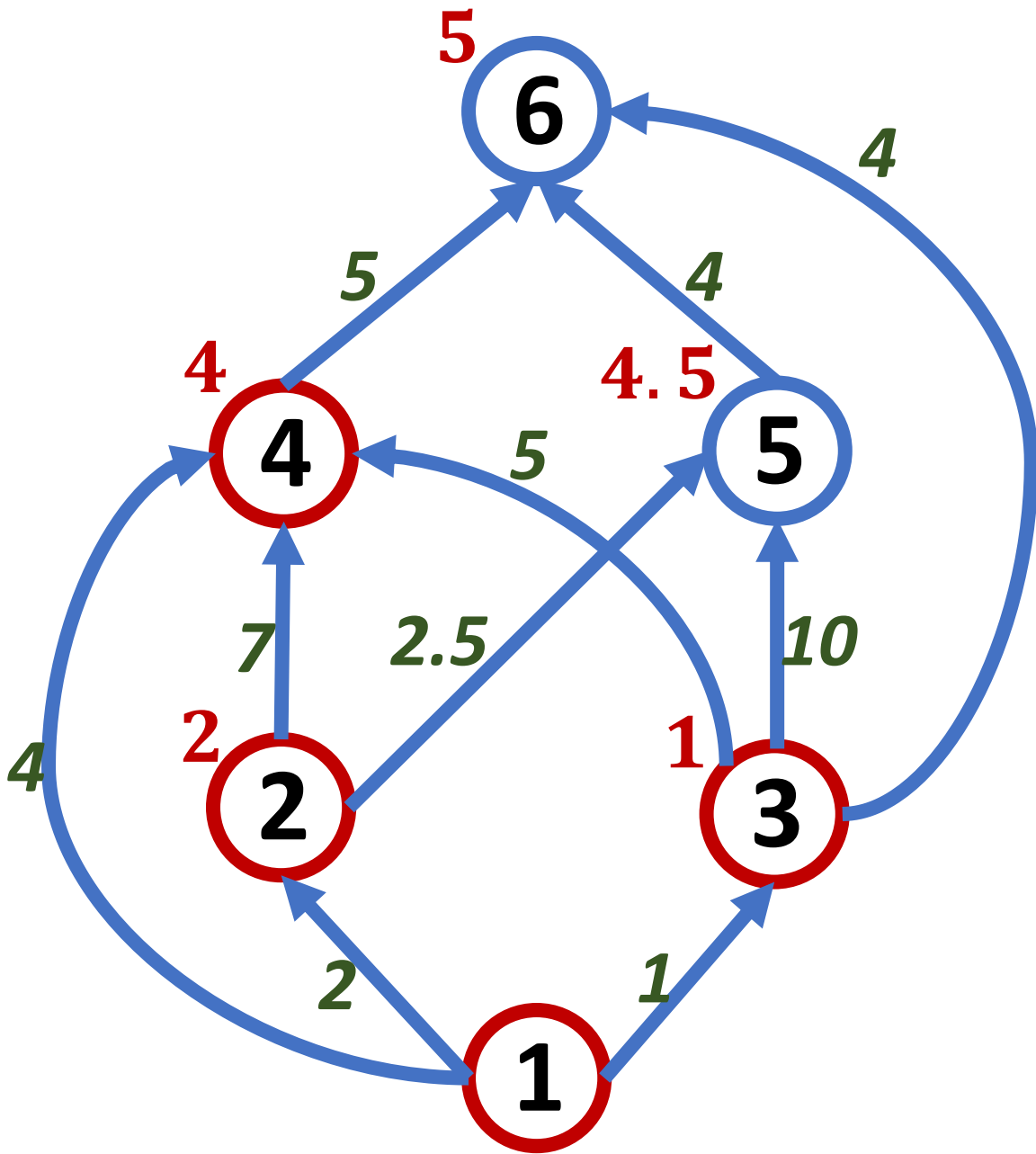
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
0	1	1	1	2	3



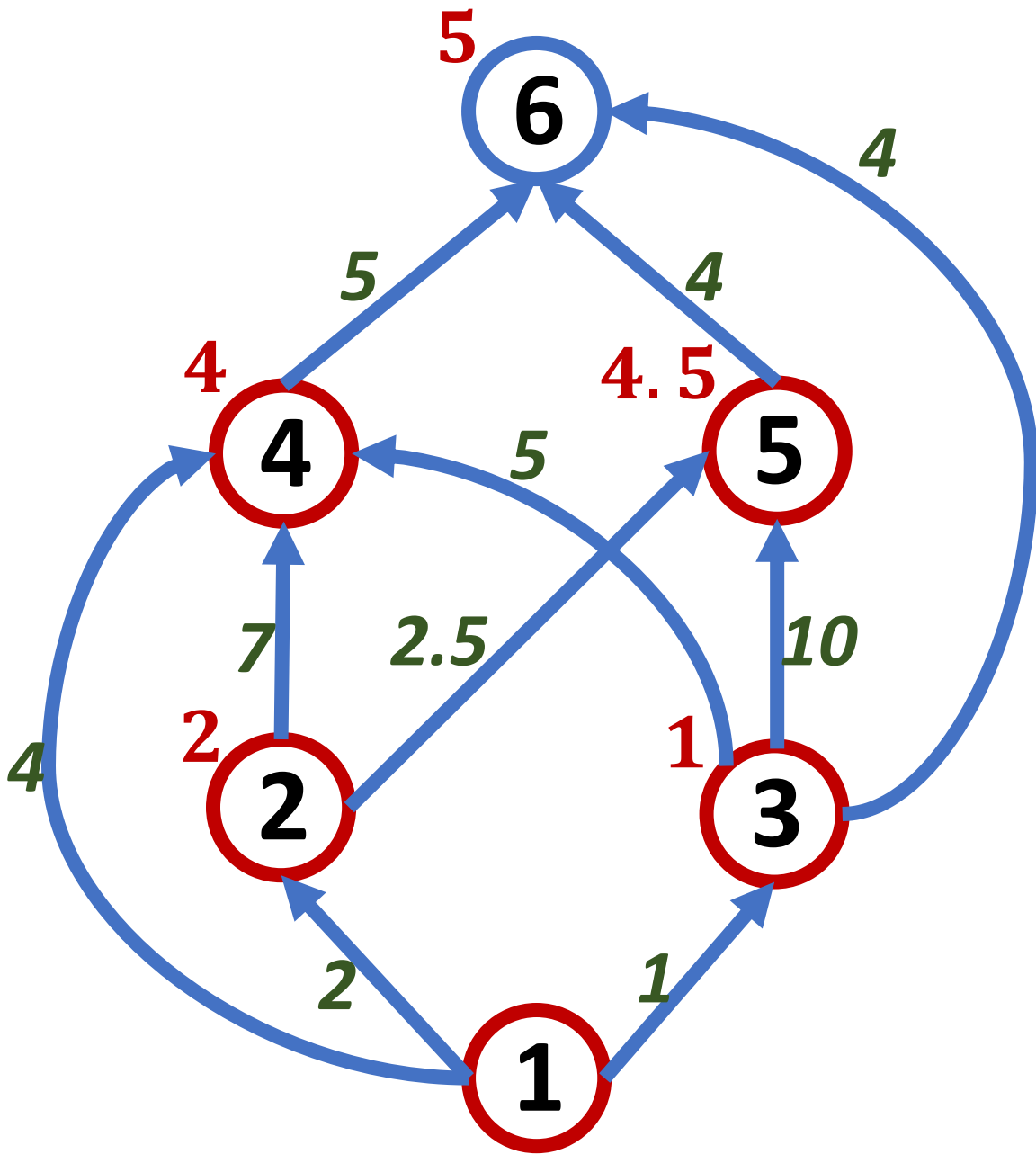
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
0	1	1	1	2	3



1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
					4+5>5
0	1	1	1	2	3

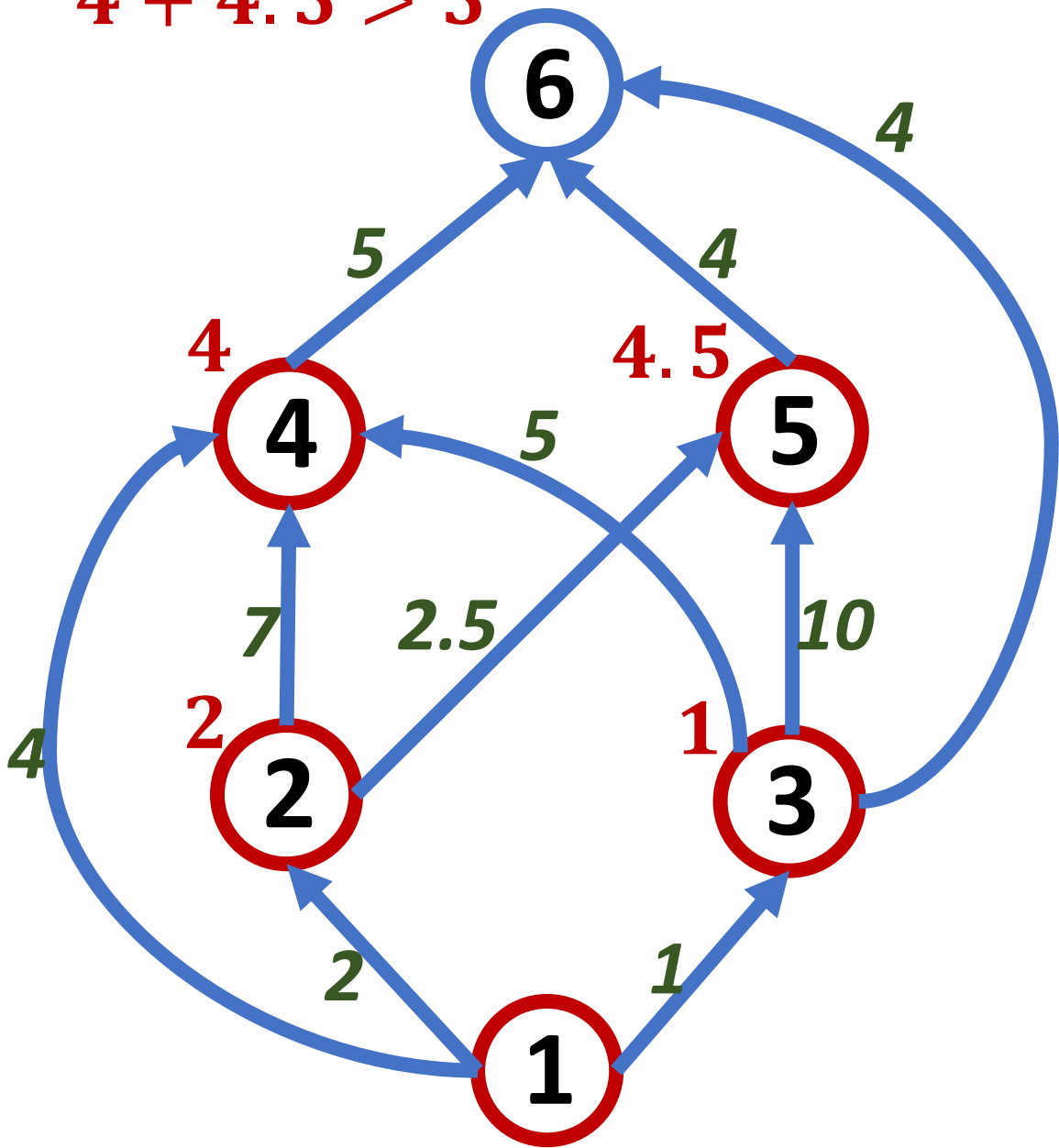


1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
				4.5	5
0	1	1	1	2	3



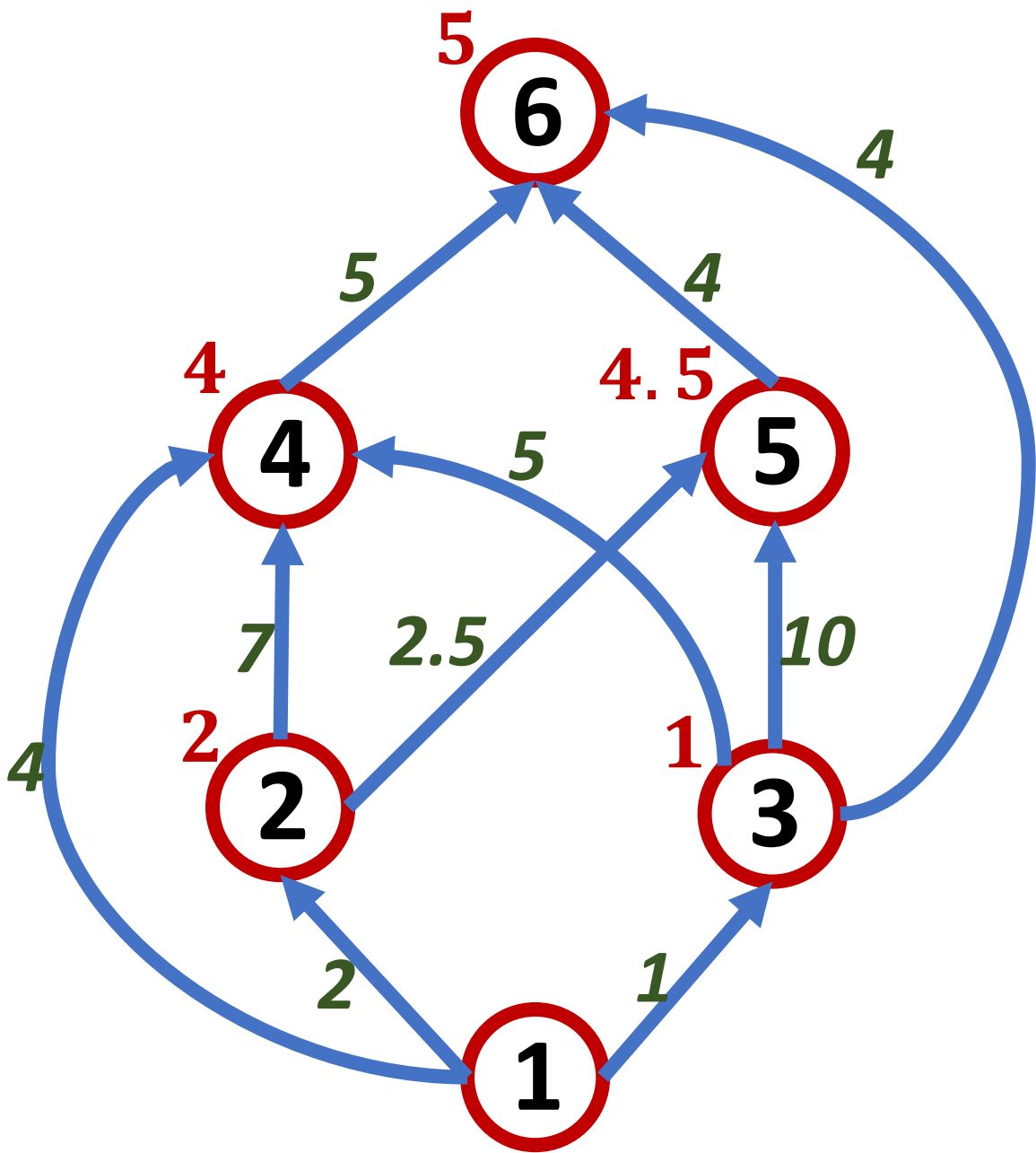
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
				4.5	5
0	1	1	1	2	3

$$4 + 4.5 > 5$$



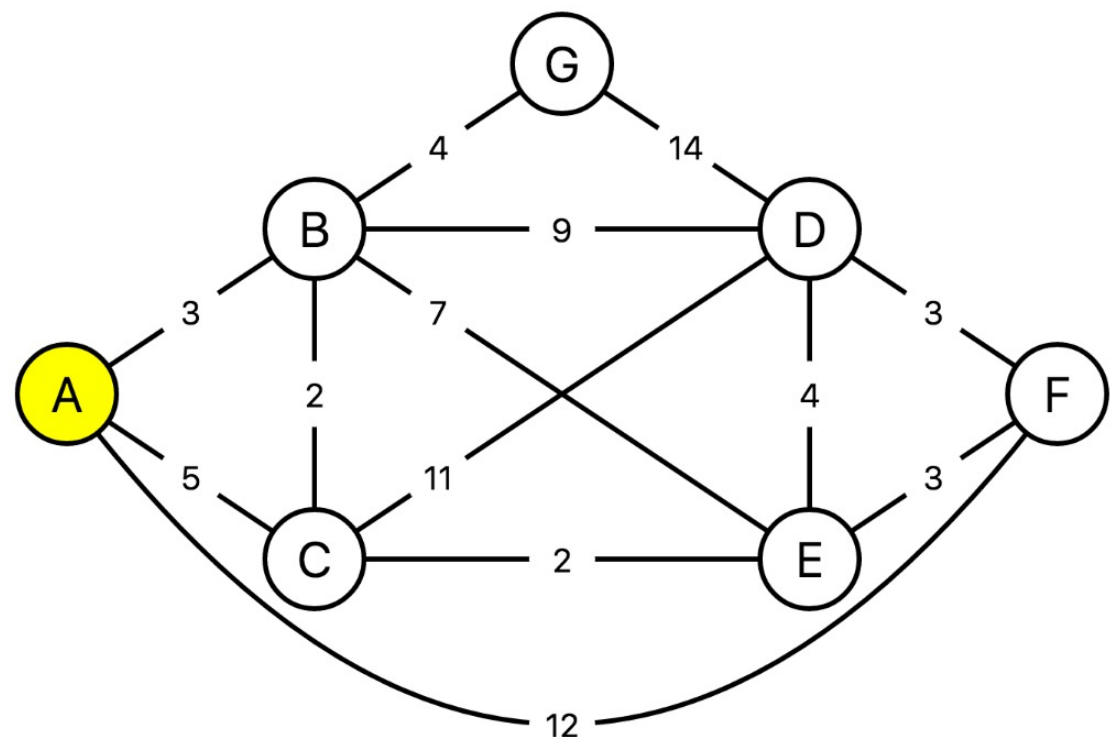
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
				4.5	5
					4+4.5 >5

0	1	1	1	2	3
----------	----------	----------	----------	----------	----------



1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
				4.5	5
					5
0	1	1	1	2	3

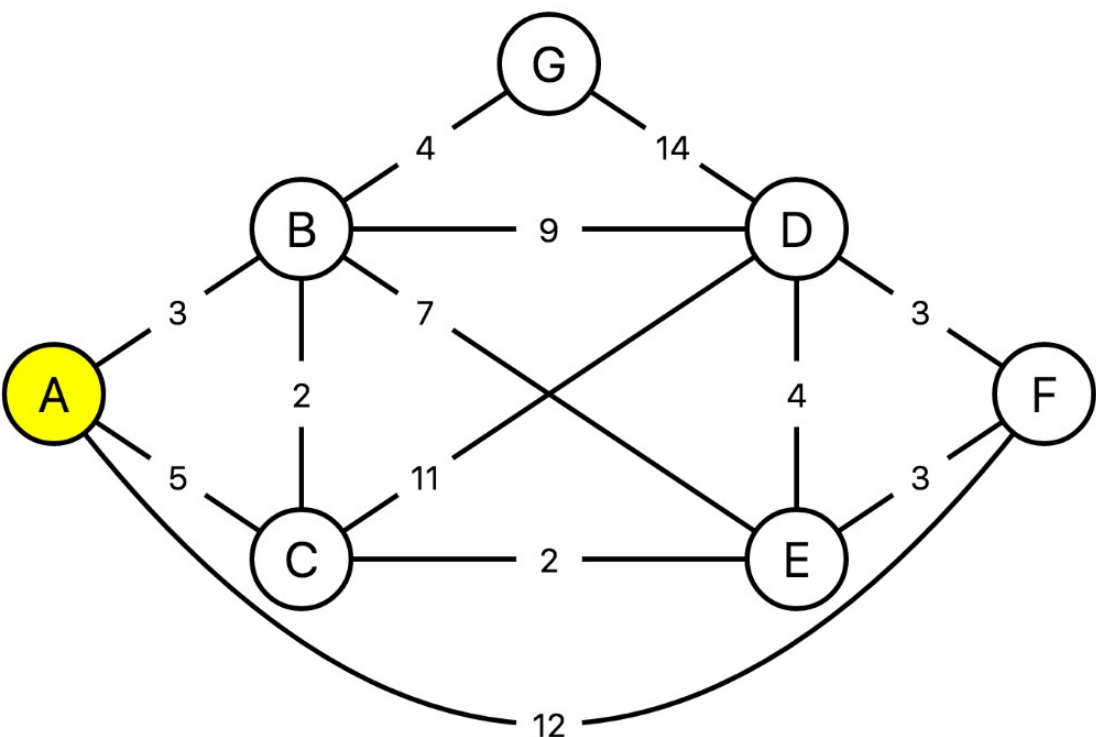
Найти кратчайший маршрут из
вершины A в F



(cs.manchester.ac.uk)

A	B	C	D	E	F	G

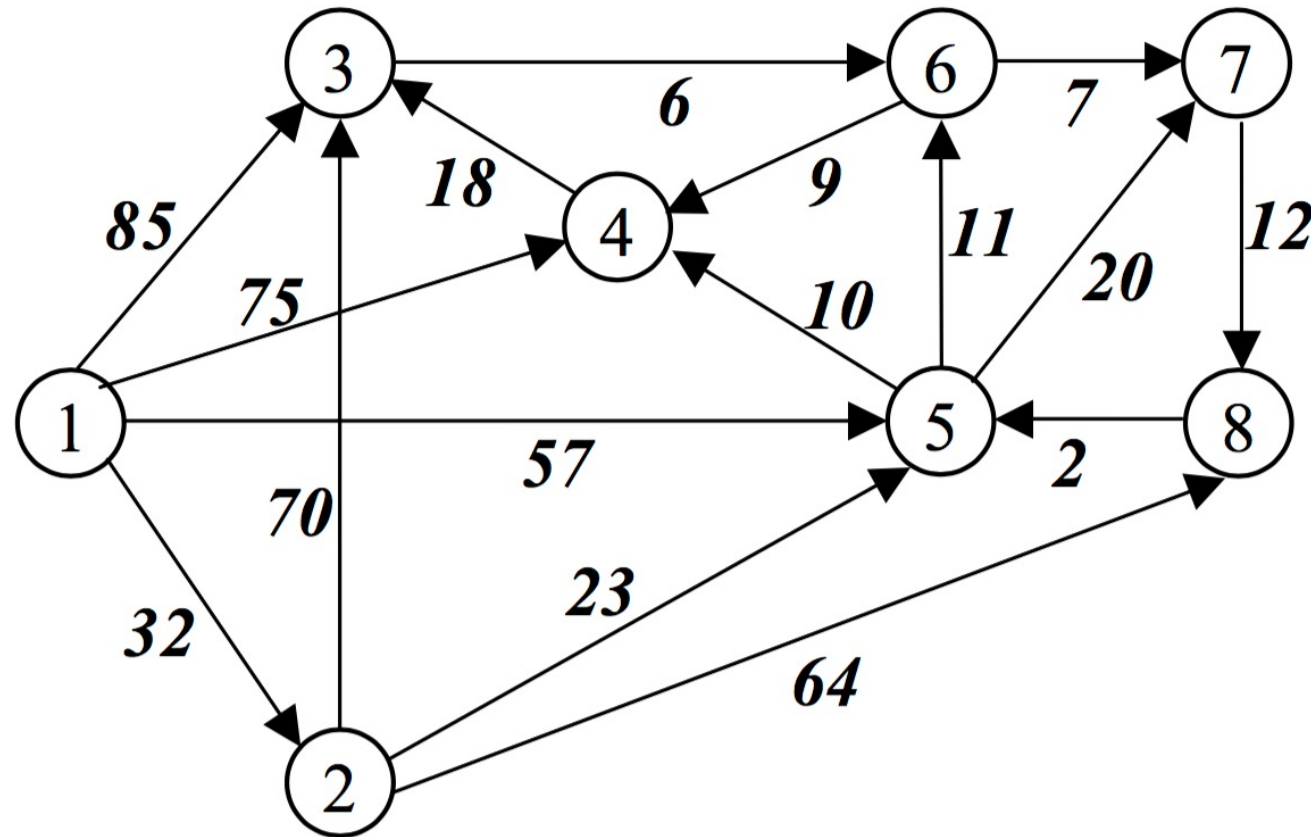
Найти кратчайший маршрут из
вершины A в F



(cs.manchester.ac.uk)

A	B	C	D	E	F	G
0	∞	∞	∞	∞	∞	∞
	3	5	∞	∞	12	∞
		5	12	10	12	7
			12	7	12	7
			11		10	7
			11		10	
-	A	A	E	C	E	B

Найти кратчайшие пути от первой вершины ко всем остальным, используя алгоритм Дейкстры.



Вычислительная сложность алгоритма Дейкстры

Напомним, что сложность алгоритма Дейкстры складывается из двух основных операций: время нахождения вершины с наименьшей величиной расстояния $d[v]$, и время совершения релаксации, т.е. время изменения величины $d[to]$.

При простейшей реализации эти операции потребуют соответственно $O(n)$ и $O(1)$ времени. Учитывая, что первая операция всего выполняется $O(n)$ раз, а вторая — $O(m)$, получаем асимптотику простейшей реализации алгоритма Дейкстры: $O(n^2 + m)$.

Простейшая реализация:
хранение в массиве,
сложность $O(n^2 + m)$

Восстановление пути:

```
vector<int> path;  
for (int v=t; v!=s; v=p[v])  
    path.push_back (v);  
path.push_back (s);  
reverse (path.begin(), path.end());
```

```
const int INF = 1000000000;  
  
int main() {  
    int n;  
    ... чтение n ...  
    vector < vector < pair<int,int> > > g (n);  
    ... чтение графа ...  
    int s = ...; // стартовая вершина  
  
    vector<int> d (n, INF), p (n);  
    d[s] = 0;  
    vector<char> u (n);  
    for (int i=0; i<n; ++i) {  
        int v = -1;  
        for (int j=0; j<n; ++j)  
            if (!u[j] && (v == -1 || d[j] < d[v]))  
                v = j;  
        if (d[v] == INF)  
            break;  
        u[v] = true;  
  
        for (size_t j=0; j<g[v].size(); ++j) {  
            int to = g[v][j].first,  
                len = g[v][j].second;  
            if (d[v] + len < d[to]) {  
                d[to] = d[v] + len;  
                p[to] = v;  
            }  
        }  
    }  
}
```

Вычислительная сложность алгоритма Дейкстры

При простейшей реализации эти операции потребуют соответственно $O(n)$ и $O(1)$ времени. Учитывая, что первая операция всего выполняется $O(n)$ раз, а вторая — $O(m)$, получаем асимптотику простейшей реализации алгоритма Дейкстры: $O(n^2 + m)$.

Понятно, что эта асимптотика является оптимальной для плотных графов, т.е. когда $m \approx n^2$. Чем более разрежен граф (т.е. чем меньше m по сравнению с максимальным количеством рёбер n^2), тем менее оптимальной становится эта оценка, и по вине первого слагаемого. Таким образом, надо улучшать время выполнения операций первого типа, не сильно ухудшая при этом время выполнения операций второго типа.

Для этого надо использовать различные вспомогательные структуры данных. Наиболее привлекательными являются **Фибоначчиевы кучи**, которые позволяют производить операцию первого вида за $O(\log n)$, а второго — за $O(1)$. Поэтому при использовании Фибоначчиевых куч время работы алгоритма Дейкстры составит $O(n \log n + m)$, что является практически теоретическим минимумом для алгоритма

Фибоначчиевы кучи

[Кратко] Фибоначчиева куча — набор из подвешенных деревьев, удовлетворяющих свойству: каждый предок не больше своих детей.

Недостатки:

- Большое потребление памяти на узел(минимум 21 байт)
- Большая константа времени работы, что делает ее малоприменимой для реальных задач
- Некоторые операции в худшем случае могут работать за $O(n)$ времени

Достоинства:

- Одно из лучших асимптотических времен работы для всех операций

Пример реализации данной структуры, состоящий из 290 строк...

<http://www.everfall.com/paste/id.php?696pwn56g065>

Вычислительная сложность алгоритма Дейкстры

В качестве компромисса можно использовать структуры данных, позволяющие выполнять **оба типа операций** (фактически, это извлечение минимума и обновление элемента) за $O(\log n)$. Тогда время работы алгоритма Дейкстры составит:

$$O(n \log n + m \log n) = O(m \log n)$$

В качестве такой структуры данных программистам на C++ удобно взять стандартный контейнер `set` или `priority_queue`. Первый основан на красно-чёрном дереве, второй — на бинарной куче. Поэтому `priority_queue` имеет меньшую константу, скрытую в асимптотике, однако у него есть и недостаток: он не поддерживает операцию удаления элемента, из-за чего приходится делать "обходной манёвр", который фактически приводит к замене в асимптотике $\log n$ на $\log m$ (с точки зрения асимптотики это на самом деле ничего не меняет, но скрытую константу увеличивает).

Реализация через set:
в контейнер помещаются пары (**расстояние, номер вершины**), которые будут автоматически отсортированы.

Массив `used` нам теперь больше не нужен.

Перед выполнением каждой успешной релаксации мы сначала удаляем старую пару, а затем, после выполнения релаксации, добавляем обратно новую пару (с новым расстоянием `d[to]`)

```
const int INF = 1000000000;
```

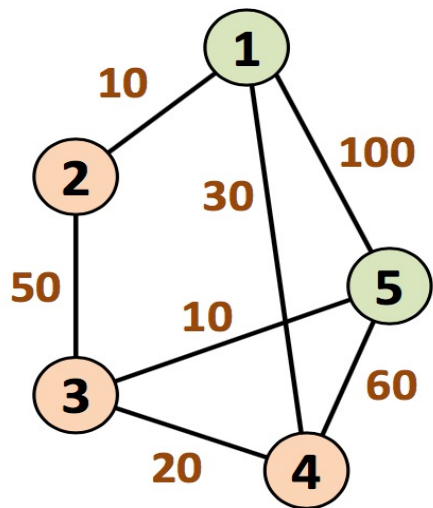
https://e-maxx.ru/algo/dijkstra_sparse

```
int main() {
    int n;
    ... чтение n ...
    vector < vector < pair<int,int> > > g (n);
    ... чтение графа ...
    int s = ...; // стартовая вершина

    vector<int> d (n, INF), p (n);
    d[s] = 0;
    set < pair<int,int> > q;
    q.insert (make_pair (d[s], s));
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase (q.begin());

        for (size_t j=0; j<g[v].size(); ++j) {
            int to = g[v][j].first,
                len = g[v][j].second;
            if (d[v] + len < d[to]) {
                q.erase (make_pair (d[to], to));
                d[to] = d[v] + len;
                p[to] = v;
                q.insert (make_pair (d[to], to));
            }
        }
    }
}
```


Найти кратчайший путь из вершины 1 в вершину 5



1	2	3	4	5	d[s]	s

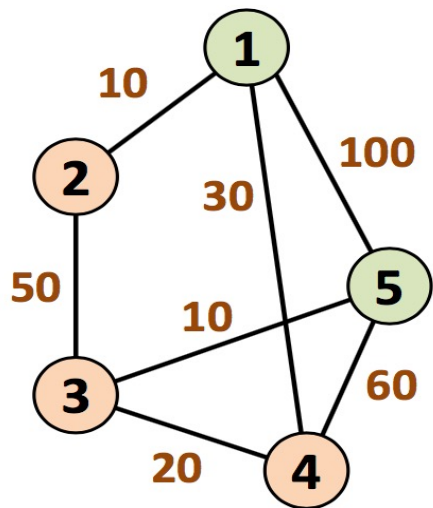
```
const int INF = 1000000000;

int main() {
    int n;
    ... чтение n ...
    vector < vector < pair<int,int> > > g (n);
    ... чтение графа ...
    int s = ...; // стартовая вершина

    vector<int> d (n, INF), p (n);
    d[s] = 0;
    set < pair<int,int> > q;
    q.insert (make_pair (d[s], s));
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase (q.begin());

        for (size_t j=0; j<g[v].size(); ++j) {
            int to = g[v][j].first,
                len = g[v][j].second;
            if (d[v] + len < d[to]) {
                q.erase (make_pair (d[to], to));
                d[to] = d[v] + len;
                p[to] = v;
                q.insert (make_pair (d[to], to));
            }
        }
    }
}
```

Найти кратчайший путь из вершины 1 в вершину 5



1	2	3	4	5	d[s]	s
0	∞	∞	∞	∞		
	10	∞	30	100		
		60	30	100		
		50		90		
				60		
0	1	4	1	3		

```
const int INF = 1000000000;
```

```
int main() {
    int n;
    ... чтение n ...
    vector < vector < pair<int,int> > > g (n);
    ... чтение графа ...
    int s = ...; // стартовая вершина

    vector<int> d (n, INF), p (n);
    d[s] = 0;
    set < pair<int,int> > q;
    q.insert (make_pair (d[s], s));
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase (q.begin());

        for (size_t j=0; j<g[v].size(); ++j) {
            int to = g[v][j].first,
                len = g[v][j].second;
            if (d[v] + len < d[to]) {
                q.erase (make_pair (d[to], to));
                d[to] = d[v] + len;
                p[to] = v;
                q.insert (make_pair (d[to], to));
            }
        }
    }
}
```


Реализация через priority_queue:

в контейнер помещаются пары (-расстояние, номер вершины).

Почему? 😊

Удалять произвольные элементы невозможно, поэтому приходится совершать "обходной манёвр": при релаксации просто не будем удалять старые пары из очереди. В результате в очереди могут находиться одновременно несколько пар для одной и той же вершины (но с разными расстояниями). Среди этих пар нас интересует только одна, для которой элемент first = d[v]. На практике версия с priority_queue оказывается несколько быстрее версии с set

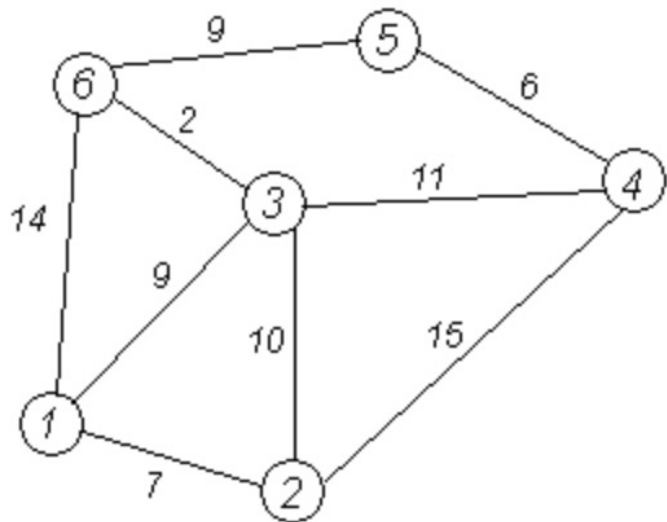
```
const int INF = 1000000000;

int main() {
    int n;
    ... чтение n ...
    vector < vector < pair<int,int> > > g (n);
    ... чтение графа ...
    int s = ...; // стартовая вершина

    vector<int> d (n, INF), p (n);
    d[s] = 0;
    priority_queue < pair<int,int> > q;
    q.push (make_pair (0, s));
    while (!q.empty()) {
        int v = q.top().second, cur_d = -q.top().first;
        q.pop();
        if (cur_d > d[v]) continue;

        for (size_t j=0; j<g[v].size(); ++j) {
            int to = g[v][j].first,
                len = g[v][j].second;
            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                p[to] = v;
                q.push (make_pair (-d[to], to));
            }
        }
    }
}
```

Найти кратчайший путь из вершины 1 в вершину 6



1	2	3	4	5	6	-d[s]	s

```

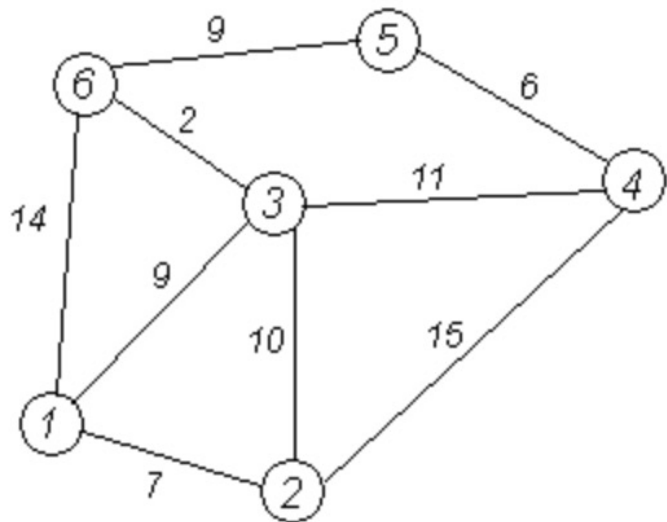
const int INF = 1000000000;

int main() {
    int n;
    ... чтение n ...
    vector < vector < pair<int,int> > > g (n);
    ... чтение графа ...
    int s = ...; // стартовая вершина

    vector<int> d (n, INF), p (n);
    d[s] = 0;
    priority_queue < pair<int,int> > q;
    q.push (make_pair (0, s));
    while (!q.empty()) {
        int v = q.top().second, cur_d = -q.top().first;
        q.pop();
        if (cur_d > d[v]) continue;

        for (size_t j=0; j<g[v].size(); ++j) {
            int to = g[v][j].first,
                len = g[v][j].second;
            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                p[to] = v;
                q.push (make_pair (-d[to], to));
            }
        }
    }
}
    
```

Найти кратчайший путь из вершины 1 в вершину 6



1	2	3	4	5	6	-d[s]	s
0	∞	∞	∞	∞	∞		
	7	9	∞	∞	14		
		9	22	∞	14		
			20	∞	11		
			20	20			
0	1	1	3	6	3		

```
const int INF = 1000000000;

int main() {
    int n;
    ... чтение n ...
    vector < vector < pair<int,int> > > g (n);
    ... чтение графа ...
    int s = ...; // стартовая вершина

    vector<int> d (n, INF), p (n);
    d[s] = 0;
    priority_queue < pair<int,int> > q;
    q.push (make_pair (0, s));
    while (!q.empty()) {
        int v = q.top().second, cur_d = -q.top().first;
        q.pop();
        if (cur_d > d[v]) continue;

        for (size_t j=0; j<g[v].size(); ++j) {
            int to = g[v][j].first,
                len = g[v][j].second;
            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                p[to] = v;
                q.push (make_pair (-d[to], to));
            }
        }
    }
}
```

Вычислительная сложность алгоритма Дейкстры

Вариант реализации алгоритма Дейкстры	Плотный граф $m = O(n^2)$	Разреженный граф $m = O(n)$
$d[i]$ – это массив (поиск за время $O(n)$)	$T = O(n^2 + m) = O(n^2)$	$T = O(n^2 + m) = O(n^2)$
$d[i]$ хранятся в бинарной куче	$T = O(n \log n + m \log n) = O(n^2 \log n)$	$T = O(n \log n + m \log n) = O(n \log n)$
$d[i]$ хранятся в Фибоначчиевой куче	$T = O(m + n \log n) = O(n^2)$	$T = O(m + n \log n) = O(n \log n)$

КОНТАКТЫ



LinkedIn

<https://ru.linkedin.com/in/mariia-gordenko-78617618b>



Telegram, Instagram

oduvan_ja



Адрес электронной почты

mgordenko@hse.ru mkgordenko@gmail.com



Личная страница

<https://www.hse.ru/staff/gordenko>